REVISION CHEAT SHEET:

DELEGATES / C#

Delegates, Action, Func, Predicate and Lambda Expressions

Interview-ready notes · examples · C# · diagrams

Topic 22 of 40

Delegates are the mechanism that allows C# to treat **behavior as data**.

Normally, you call a method directly:

C#
```
decimal total = CalculateTotal(gift);
```

With a delegate, you can store a compatible method in a variable:

C#
```
Func<Gift, decimal> calculator = CalculateTotal;
```

and invoke it later:

C#
```
decimal total = calculator(gift);
```

You can also:

EXAMPLE / FLOW
```
Pass behavior into another method

Return behavior from a method

Store behavior in an object

Choose behavior at runtime

Combine multiple callbacks

Build event systems

Use lambdas with collections and LINQ
```

A delegate is a type-safe object representing references to methods with a compatible parameter list and return type. C# verifies method compatibility when the method is assigned to the delegate.

This concept is fundamental to:

EXAMPLE / FLOW
```
Events

LINQ

ASP.NET Core configuration

Callbacks

Strategy-style programming
```

```
Task continuations

Collection filtering and sorting
```

High-Level Mental Model

Without delegates:

EXAMPLE / FLOW

```
GiftService
    ↓
Hard-coded behavior
    ↓
SendEmailReceipt()
```

With delegates:

EXAMPLE / FLOW

```
GiftService
    ↓
Receives behavior
    ↓
Action<Gift>
```

At runtime that behavior could be:

EXAMPLE / FLOW

```
SendEmailReceipt

WriteAuditLog

PublishMessage

DoNothing

TestCallback
```

Central Idea

A delegate answers:

> **What method should be executed?**

rather than:

> **What value should be stored?**

Example:

C#

```
Func<decimal, decimal> calculator;
```

This variable doesn't hold a decimal.

It holds something that can accept:

EXAMPLE / FLOW

```
decimal
```

and return:

EXAMPLE / FLOW

```
decimal
```

1. Declaring a Custom Delegate

Syntax:

C#

```
public delegate decimal GiftFeeCalculator(
    decimal giftAmount);
```

This defines a new delegate type named:

EXAMPLE / FLOW

```
GiftFeeCalculator
```

Its contract is:

EXAMPLE / FLOW

```
Input
    →
decimal

Output
    →
decimal
```

Any compatible method may be assigned to it.

Compatible Method

C#

```
public static decimal CalculateProcessingFee(
    decimal giftAmount)
{
    return giftAmount * 0.03m;
}
```

Assign it:

C#

```
GiftFeeCalculator calculator = CalculateProcessingFee;
```

Invoke:

C#

```
decimal fee = calculator(1000m);
```

Result:

EXAMPLE / FLOW

```
30
```

Delegate Flow

EXAMPLE / FLOW

```
CalculateProcessingFee method
        ↓
Method reference
        ↓
GiftFeeCalculator variable
        ↓
calculator(1000m)
        ↓
Method executes
        ↓
30
```

2. Method Group Conversion

Notice:

C#

```
GiftFeeCalculator calculator = CalculateProcessingFee;
```

We did **not** write:

C#

```
CalculateProcessingFee();
```

Why?

Because:

C#

```
CalculateProcessingFee
```

is being used as a **method group**.

We want:

EXAMPLE / FLOW

```
Reference to method
```

not:

EXAMPLE / FLOW

```
Call method immediately
```

Wrong Mental Model

C#

```
GiftFeeCalculator calculator = CalculateProcessingFee();
```

This would attempt to:

EXAMPLE / FLOW
```
Call method now
```

and assign its returned:

EXAMPLE / FLOW
```
decimal
```

to a delegate variable.

Those are incompatible concepts.

Correct

C#
```
GiftFeeCalculator calculator = CalculateProcessingFee;
```

3. Delegate Invocation

A delegate can be invoked using normal call syntax:

C#
```
decimal fee = calculator(1000m);
```

or explicitly:

C#
```
decimal fee = calculator.Invoke(1000m);
```

These communicate the same basic invocation.

Most C# code prefers:

C#
```
calculator(1000m);
```

because it looks like a normal method call.

Nullable Delegates

A delegate variable may be nullable:

C#
```
Action<Gift>? callback = null;
```

Calling it directly:

C#
```
callback(gift);
```

would risk:

EXAMPLE / FLOW
```
NullReferenceException
```

You can use:

```C#
callback?.Invoke(
    gift);
```

Now invocation occurs only when a delegate exists.

4. Static Method Delegate

A delegate may refer to a static method.

```C#
public static void LogGift(
    Gift gift)
{
    Console.WriteLine(
        gift.GiftId);
}
```

Assign:

```C#
Action<Gift> action = LogGift;
```

The delegate needs the method information.

There is no target object instance involved.

5. Instance Method Delegate

A delegate can also reference an instance method.

```C#
public sealed class GiftLogger
{
    public void Log(
        Gift gift)
    {
        Console.WriteLine(
            gift.GiftId);
    }
}
```

Usage:

```C#
GiftLogger logger = new();

Action<Gift> action = logger.Log;
```

The delegate now effectively contains:

```EXAMPLE / FLOW
Target object
    →
logger

Method
    →
Log
```

A delegate that refers to an instance method keeps both the method and the target instance needed to invoke it.

Delegate Instance Mental Model

EXAMPLE / FLOW

```
Delegate
    |
    +-- Method
    |
    +-- Target object
         if instance method
```

6. Passing Methods as Arguments

This is one of the most important uses of delegates.

Suppose:

C#

```
public static void ProcessGift(
    Gift gift,
    Action<Gift> onProcessed)
{
    // Process gift.

    onProcessed(gift);
}
```

Now caller chooses the callback.

C#

```
ProcessGift(
    gift,
    LogGift);
```

Another caller:

C#

```
ProcessGift(
    gift,
    SendReceipt);
```

Same processing method.

Different behavior supplied by caller.

Callback Mental Model

EXAMPLE / FLOW

```
Caller
    ↓
passes method
    ↓
ProcessGift
    ↓
does work
    ↓
invokes callback
```

This is:

EXAMPLE / FLOW

```
Callback programming
```

7. Why Built-In Delegates Exist

Creating custom delegates for everything would become repetitive:

C#

```
public delegate void GiftLogger(
    Gift gift);

public delegate void CampaignLogger(
    Campaign campaign);

public delegate bool GiftFilter(
    Gift gift);

public delegate decimal GiftCalculator(
    Gift gift);
```

.NET therefore provides common generic delegate families:

EXAMPLE / FLOW

```
Action

Func

Predicate
```

`Action` and `Func` cover most everyday delegate signatures, with versions supporting up to sixteen input parameters. `Action` represents behavior returning `void`; `Func` represents behavior returning a value.

8. Action

`Action` represents a method that:

EXAMPLE / FLOW

```
Returns void
```

Action with No Parameters

C#

```
Action action = SayHello;
```

Method:

C#

```
static void SayHello()
{
    Console.WriteLine(
        "Hello");
}
```

Call:

```C#
action();
```

Action<T>

One input:

```C#
Action<Gift> printGift = PrintGift;
```

Method:

```C#
static void PrintGift(
    Gift gift)
{
    Console.WriteLine(
        gift.GiftId);
}
```

`Action<T>` represents a method receiving `T` and returning `void`.

Action<T1,T2>

```C#
Action<Gift, string> log = LogGift;
```

Compatible method:

```C#
static void LogGift(
    Gift gift,
    string message)
{
    Console.WriteLine(
        $"{gift.GiftId}: {message}");
}
```

Action Mental Model

```EXAMPLE / FLOW
Input
    ↓
Perform behavior
    ↓
No returned value
```

Use `Action` when the behavior means:

```EXAMPLE / FLOW
Do something
```

rather than:

```EXAMPLE / FLOW
Calculate something and return it
```

Common Action Examples

```C#
Action log;

Action<Gift> processGift;

Action<Gift, Campaign> associate;

Action<string, Exception> logError;
```

9. Func

`Func` represents a method that:

EXAMPLE / FLOW

```
Returns a value
```

The **last generic type parameter is always the return type**.

Func<TResult>

No input parameters:

```C#
Func<DateTime> getCurrentTime = GetCurrentTime;
```

Method:

```C#
static DateTime GetCurrentTime()
{
    return DateTime.UtcNow;
}
```

Func<T,TResult>

```C#
Func<Gift, decimal> amountSelector = GetGiftAmount;
```

Method:

```C#
static decimal GetGiftAmount(
    Gift gift)
{
    return gift.TotalAmount;
}
```

Read it:

EXAMPLE / FLOW

```
Input
    →
Gift

Output
```

```
    →
decimal
```

Func<T1,T2,TResult>

```C#
Func<decimal, decimal, decimal>
    add = Add;
```

Method:

```C#
static decimal Add(
    decimal first,
    decimal second)
{
    return first + second;
}
```

Reading Func Correctly

```C#
Func<Gift, bool>
```

means:

```EXAMPLE / FLOW
Gift
    →
bool
```

```C#
Func<Gift, decimal>
```

means:

```EXAMPLE / FLOW
Gift
    →
decimal
```

```C#
Func<Gift, Campaign, bool>
```

means:

```EXAMPLE / FLOW
Gift
Campaign
    ↓
bool
```

The final generic argument is always the result.

Action vs Func

Delegate	Inputs	Return
`Action`	0	`void`
`Action<T>`	1	`void`
`Action<T1,T2>`	2	`void`
`Func<TResult>`	0	`TResult`
`Func<T,TResult>`	1	`TResult`
`Func<T1,T2,TResult>`	2	`TResult`

Simple Memory Rule

EXAMPLE / FLOW

```
Action
    →
ACT

Func
    →
FUNCTION returning a value
```

10. Predicate<T>

`Predicate<T>` represents:

EXAMPLE / FLOW

```
T
    →
bool
```

Its conceptual signature is:

C#

```
public delegate bool Predicate<T>(
    T obj);
```

The actual modern framework declaration is contravariant in `T`.

Example:

C#

```
Predicate<Gift> isLargeGift = IsLargeGift;
```

Method:

C#

```
static bool IsLargeGift(
    Gift gift)
{
    return gift.TotalAmount >= 1000m;
}
```

Predicate Meaning

A predicate answers:

EXAMPLE / FLOW

```
Does this value satisfy
a condition?
```

Result:

EXAMPLE / FLOW

```
true

or

false
```

Predicate Example with List

From Topic 21:

C#

```
gifts.RemoveAll(
    gift =>
        gift.Status
        == GiftStatus.Cancelled);
```

`List<T>.RemoveAll` accepts a:

C#

```
Predicate<T>
```

For:

C#

```
List<Gift>
```

that becomes:

C#

```
Predicate<Gift>
```

Predicate vs Func<T,bool>

These have equivalent broad shapes:

C#

```
Predicate<Gift>
```

and:

C#

```
Func<Gift, bool>
```

Both accept:

EXAMPLE / FLOW

```
Gift
```

DELEGATES / C#

and return:

EXAMPLE / FLOW

```
bool
```

But they are **different delegate types**.

Some APIs specifically require:

C#

```
Predicate<T>
```

while LINQ commonly uses:

C#

```
Func<T, bool>
```

11. Custom Delegate vs Action/Func

Use:

C#

```
Action<Gift>
```

when the signature alone clearly explains the purpose.

Use a custom delegate when a semantic name improves the API.

Example:

C#

```
public delegate bool
    GiftEligibilityRule(
        Gift gift,
        Campaign campaign);
```

Compare:

C#

```
Func<Gift, Campaign, bool>
```

Both work.

But:

EXAMPLE / FLOW

```
GiftEligibilityRule
```

tells the caller what the behavior means.

Custom Delegate Benefits

A custom type may provide:

EXAMPLE / FLOW

```
Domain meaning

Clear documentation

Distinct API contract

Support for signatures that don't fit
Action/Func conveniently
```

Examples include delegates using:

EXAMPLE / FLOW

```
ref

out
```

or specialized domain semantics.

12. Lambda Expressions

A lambda creates an anonymous function using:

C#

```
=>
```

Microsoft documents two primary forms:

EXAMPLE / FLOW

```
Expression lambda

Statement lambda
```

and lambdas can convert to compatible delegate types or expression-tree types.

Expression Lambda

C#

```
Func<Gift, bool> isLargeGift = gift => gift.TotalAmount >= 1000m;
```

General form:

C#

```
parameters => expression
```

Statement Lambda

C#

```
Action<Gift> print =
    gift =>
    {
        Console.WriteLine(
            $"Gift: {gift.GiftId}");

        Console.WriteLine(
```

```
            $"Amount: {gift.TotalAmount}");
};
```


General form:

```C#
parameters =>
{
    statements
}
```

One Parameter

Parentheses may be omitted:

```C#
gift =>
    gift.TotalAmount > 100m
```

Multiple Parameters

Use parentheses:

```C#
(decimal amount, decimal rate) =>
    amount * rate
```

Usually parameter types can be inferred:

```C#
(amount, rate) =>
    amount * rate
```

when the target delegate provides enough type information.

Zero Parameters

```C#
Action action = () => Console.WriteLine("Processing");
```

Lambda Type Inference

Example:

```C#
Func<Gift, bool> filter = gift => gift.TotalAmount >= 500m;
```

Because target type is:

```C#
Func<Gift, bool>
```

compiler knows:

EXAMPLE / FLOW

```
gift
    →
Gift
```

and return must be convertible to:

EXAMPLE / FLOW

```
bool
```

Lambda parameter and return compatibility are checked against the delegate target.

Natural Lambda Types

Modern C# can infer a delegate type in some cases:

C#

```
var parse = (string value) => int.Parse(value);
```

The compiler can infer:

C#

```
Func<string, int>
```

When there isn't enough type information, explicit delegate typing is still required.

13. Named Method vs Lambda

Named method:

C#

```
static bool IsLargeGift(
    Gift gift)
{
    return gift.TotalAmount >= 1000m;
}
```

Usage:

C#

```
Predicate<Gift> predicate = IsLargeGift;
```

Lambda:

C#

```
Predicate<Gift> predicate = gift => gift.TotalAmount >= 1000m;
```

When Named Methods Are Better

Prefer a named method when behavior is:

EXAMPLE / FLOW

```
Complex

Reused

Worth testing independently
```

```
Important to domain language

Long enough to deserve a name
```

When Lambdas Are Better

Lambdas work well for:

EXAMPLE / FLOW

```
Short local behavior

Simple filters

Simple projections

Callbacks used once

LINQ expressions
```

Avoid Giant Lambdas

Hard to read:

C#

```
ProcessGift(
    gift =>
    {
        // 80 lines
        // validation
        // persistence
        // API calls
        // logging
    });
```

At that point, extract a named method or service.

14. Anonymous Methods

Before lambdas became common, C# supported:

C#

```
delegate
```

anonymous-method syntax.

Example:

C#

```
Action<Gift> print = delegate (Gift gift) { Console.WriteLine(gift.GiftId);
    };
```

Modern equivalent:

C#

```
Action<Gift> print = gift => Console.WriteLine(gift.GiftId);
```

Lambda syntax is usually clearer and more common in modern C#.

15. Passing Lambdas to Methods

Method:

```C#
public static void Process(
    Gift gift,
    Action<Gift> callback)
{
    callback(
        gift);
}
```

Call:

```C#
Process(
    gift,
    current =>
    {
        Console.WriteLine(
            current.GiftId);
    });
```

No separate callback method is required.

16. Returning Delegates

Methods can also return behavior.

Example:

```C#
public static Func<Gift, bool>
    CreateMinimumAmountFilter(
        decimal minimumAmount)
{
    return gift =>
        gift.TotalAmount >= minimumAmount;
}
```

Usage:

```C#
Func<Gift, bool> filter = CreateMinimumAmountFilter(1000m);
```

Then:

```C#
bool eligible = filter(gift);
```

This Introduces Closures

The lambda uses:

```C#
minimumAmount
```

which belongs to the surrounding method.

The lambda captures it.

17. Closures

A lambda can use variables declared outside itself.

Example:

```C#
decimal minimumAmount = 1000m;

Func<Gift, bool> isLarge = gift => gift.TotalAmount >= minimumAmount;
```

The lambda captures:

```EXAMPLE / FLOW
minimumAmount
```

This is called a:

```EXAMPLE / FLOW
Closure
```

Captured variables can remain alive for as long as the delegate that needs them remains reachable.
Multiple lambdas capturing the same variable can observe later changes to that captured variable.

Closure Mental Model

```EXAMPLE / FLOW
Outer variable
minimumAmount
       ↓
Captured by
       ↓
Lambda
       ↓
Delegate keeps required
capture state alive
```

Captured Variable Is Not Necessarily a Snapshot

Consider:

```C#
decimal minimum = 100m;

Func<Gift, bool> filter = gift => gift.TotalAmount >= minimum;

minimum = 1000m;
```

Later:

```C#
bool result = filter(gift);
```

uses the current captured:

EXAMPLE / FLOW

```
1000
```

not automatically:

EXAMPLE / FLOW

```
100
```

The lambda captures the variable/state, not simply an immutable snapshot of the original value.

Capturing Loop Variables

Be careful with variables that continue changing.

C#

```
List<Action> actions = [];

for (
    int i = 0;
    i < 3;
    i++)
{
    actions.Add(
        () =>
            Console.WriteLine(
                i));
}
```

The callbacks capture the same evolving loop variable.

By the time they run, they may all observe its final value.

A safer intent can be made explicit:

C#

```
for (
    int i = 0;
    i < 3;
    i++)
{
    int copy = i;

    actions.Add(
        () =>
            Console.WriteLine(
                copy));
}
```

Now each callback captures a separate local value.

Closure Performance

A non-capturing lambda can often be implemented without creating per-call capture state.

A capturing lambda may require compiler-generated storage to keep captured variables alive.

Therefore, in very hot code:

EXAMPLE / FLOW

```
Capturing lambda
```

can have additional allocation/lifetime cost compared with:

```
EXAMPLE / FLOW
Static method

Static lambda

Non-capturing callback
```

Don't optimize this blindly, but understand it when profiling allocation-heavy code.

18. Static Lambdas

You can write:

```
C#
Func<Gift, decimal> selector = static gift => gift.TotalAmount;
```

A static lambda cannot capture local variables or instance state from its surrounding context.

Example Invalid Capture

Conceptually invalid:

```
C#
decimal multiplier = 2m;

Func<decimal, decimal> calculate = static value => value * multiplier;
```

because:

```
EXAMPLE / FLOW
multiplier
```

belongs to the enclosing scope.

Why Static Lambda Helps

It communicates:

```
EXAMPLE / FLOW
This callback does not depend on
hidden captured local state.
```

It can also help avoid accidental closure allocations.

19. Delegates as Strategy Behavior

Suppose receipt formatting can vary.

Without delegate:

```
C#
if (
    type
    ==
    "Email")
{
    ...
```

```
}
else if (
    type
    ==
    "Print")
{
    ...
}
```

Delegate:

```C#
Func<Gift, string> formatter;
```

Choose behavior:

```C#
formatter = FormatEmailReceipt;
```

or:

```C#
formatter = FormatPrintedReceipt;
```

Then:

```C#
string receipt = formatter(gift);
```

Strategy Flow

EXAMPLE / FLOW

```
Choose behavior
    ↓
Store delegate
    ↓
Later invoke
```

This is a lightweight form of strategy programming.

For larger stateful strategies, interfaces/classes may be cleaner.

20. Multicast Delegates

A delegate may contain multiple methods in its:

EXAMPLE / FLOW

```
Invocation List
```

Example:

```C#
Action<Gift> callbacks = LogGift;

callbacks += SendReceipt;

callbacks += PublishAudit;
```

Invoke:


```C#
callbacks(
    gift);
```

Methods execute synchronously in invocation-list order.

Invocation Flow

EXAMPLE / FLOW

```
callbacks(gift)
    ↓
LogGift(gift)
    ↓
SendReceipt(gift)
    ↓
PublishAudit(gift)
```

Combining Delegates

You may write:

```C#
Action<Gift> first = LogGift;

Action<Gift> second = SendReceipt;

Action<Gift> combined = first + second;
```

The original delegates remain unchanged; combining them creates another delegate whose invocation list contains the combined entries.

+=

Common syntax:

```C#
callbacks += SendReceipt;
```

Removing a Handler

```C#
callbacks -= SendReceipt;
```

The matching entry is removed from the invocation list.

If the same delegate was added multiple times, removal removes one matching occurrence, specifically the rightmost matching entry.

Duplicate Delegate Entries

This is legal:

```
C#
callbacks += LogGift;

callbacks += LogGift;
```

Then invocation calls:

```
EXAMPLE / FLOW
LogGift

LogGift
```

twice.

Multicast Return Values

Suppose:

```
C#
Func<int> handlers;
```

contains:

```
EXAMPLE / FLOW
Method A → returns 10

Method B → returns 20

Method C → returns 30
```

Calling:

```
C#
int result = handlers();
```

returns the value from the **last invoked method**:

```
EXAMPLE / FLOW
30
```

The earlier return values aren't automatically aggregated.

Therefore

Multicast delegates are easiest to reason about with:

```
C#
Action
```

because there is no return-value ambiguity.

Exceptions in Multicast Delegates

Suppose invocation list is:

EXAMPLE / FLOW

```
A

B

C
```

and:

EXAMPLE / FLOW

```
B throws
```

without handling the exception.

Then:

EXAMPLE / FLOW

```
A executes

B throws

C does not execute
```

The exception propagates to the delegate caller.

Important

Multicast delegate invocation is **not** automatically:

EXAMPLE / FLOW

```
Parallel

Independent

Exception-isolated
```

It is synchronous ordered invocation.

21. GetInvocationList

You can inspect handlers:

C#

```
Delegate[] handlers = callbacks.GetInvocationList();
```

Advanced code can invoke each separately:

C#

```
foreach (
    Action<Gift> handler
    in callbacks.GetInvocationList())
{
    try
    {
        handler(
            gift);
    }
    catch (
        Exception ex)
    {
```

```
        // Per-handler policy.
    }
}
```

DELEGATES / C#

This is useful when:

EXAMPLE / FLOW

```
One callback failure
should not prevent
all later callbacks
```

But such behavior should be an intentional design decision.

22. Delegate Variance

Delegates support covariance and contravariance in compatible method assignment.

Suppose:

C#

```
public class Entity
{
}

public class Gift :
    Entity
{
}
```

A delegate expecting:

EXAMPLE / FLOW

```
Gift → Entity
```

can sometimes bind a method that accepts a less-derived parameter or returns a more-derived result. C# supports this method-signature variance for delegates.

Return-Type Covariance

Delegate:

C#

```
public delegate Entity
    EntityFactory();
```

Method:

C#

```
static Gift CreateGift()
{
    return new Gift();
}
```

This can be compatible because:

EXAMPLE / FLOW

```
Gift
IS AN
Entity
```

A method returning something more specific is safe where an Entity is expected.

Parameter Contravariance

Delegate:

```C#
public delegate void
    GiftProcessor(
        Gift gift);
```

Method:

```C#
static void ProcessEntity(
    Entity entity)
{
}
```

The method can process any `Entity`, therefore it can certainly process a `Gift`.

Variance in Action and Func

`Action<T>` uses a contravariant input parameter:

```
EXAMPLE / FLOW
in T
```

while `Func<T,TResult>` uses:

```
EXAMPLE / FLOW
in T
```

for inputs and:

```
EXAMPLE / FLOW
out TResult
```

for its result.

Producer / Consumer Mental Model

```
EXAMPLE / FLOW
Func output
    →
out
    →
covariance
```

```
EXAMPLE / FLOW
Action input
    →
in
    →
contravariance
```

This builds directly on Topic 20.

23. Delegates and Collection APIs

From Topic 21:

```C#
gifts.RemoveAll(
    gift =>
        gift.Status
        == GiftStatus.Cancelled);
```

The list receives behavior.

Conceptually:

```EXAMPLE / FLOW
For each Gift
    ↓
invoke predicate
    ↓
true?
    →
remove
```

Sort Callback

```C#
gifts.Sort(
    (left, right) =>
        left.TotalAmount.CompareTo(
            right.TotalAmount));
```

The collection does not need hard-coded knowledge of:

```EXAMPLE / FLOW
How Gifts should be ordered
```

The caller supplies comparison behavior.

Find

```C#
Gift? gift = gifts.Find(current => current.GiftId == giftId);
```

Again:

```EXAMPLE / FLOW
Collection algorithm
+
Caller-provided behavior
```

This pattern is everywhere in modern C#.

24. Delegates and LINQ Preview

LINQ uses delegates extensively.

Example:

```C#
IEnumerable<Gift> largeGifts = gifts.Where(gift => gift.TotalAmount >= 1000m);
```

For LINQ-to-Objects:

```C#
Where
```

receives behavior shaped like:

```C#
Func<Gift, bool>
```

Lambda usage with standard query operators is a major delegate use case.

LINQ is Topic 24.

25. Delegate vs Expression Tree

These may look identical:

```C#
Func<Gift, bool> filter = gift => gift.TotalAmount > 1000m;
```

and:

```C#
Expression<Func<Gift, bool>>
    filterExpression = gift => gift.TotalAmount > 1000m;
```

But they represent different things.

Delegate

```EXAMPLE / FLOW
Executable behavior
```

Call it:

```C#
bool result = filter(gift);
```

Expression Tree

```EXAMPLE / FLOW
Data structure describing code
```

A library can inspect:

```EXAMPLE / FLOW
Gift

TotalAmount

>
```

```
1000
```


and possibly translate it into another representation such as SQL.

Microsoft documents that a lambda may convert either to a delegate or to an `Expression<TDelegate>` depending on the target type.

This distinction becomes very important with:

EXAMPLE / FLOW

```
Enumerable.Where
```

versus:

EXAMPLE / FLOW

```
Queryable.Where
```

in Topic 24.

26. Delegates and Events Preview

An event is built on delegate concepts.

Delegate alone:

C#

```
public Action<Gift>? GiftProcessed;
```

external code could potentially:

EXAMPLE / FLOW

```
Subscribe

Replace delegate

Invoke delegate
```

depending on accessibility.

An event:

C#

```
public event Action<Gift>?
    GiftProcessed;
```

restricts outside code primarily to subscription/unsubscription.

Only the declaring type normally raises it.

That is why events provide better publisher/subscriber encapsulation.

Topic 23 covers this in depth.

27. Async Delegates

Delegates can represent asynchronous methods.

Prefer:

```
C#
Func<Gift, Task>
```

for asynchronous behavior that doesn't return a result.

Example:

```
C#
Func<Gift, Task> sendReceiptAsync = SendReceiptAsync;
```

Method:

```
C#
static async Task SendReceiptAsync(
    Gift gift)
{
    await Task.Delay(
        100);

    Console.WriteLine(
        gift.GiftId);
}
```

Call:

```
C#
await sendReceiptAsync(
    gift);
```

Async Result

```
C#
Func<Gift, Task<decimal>>
    calculateAsync;
```

Read:

```
EXAMPLE / FLOW
Gift
    ↓
Task<decimal>
```

Avoid async Action for Ordinary Async Work

This is dangerous:

```
C#
Action<Gift> action =
    async gift =>
    {
        await SendReceiptAsync(
            gift);
    };
```

Because `Action<Gift>` returns:

EXAMPLE / FLOW

```
void
```

the async lambda effectively becomes:

EXAMPLE / FLOW

```
async void
```

That cannot be awaited by the caller, and exceptions don't follow normal `Task` observation rules. .NET guidance recommends `async void` primarily for event handlers and `Task`/`Task<T>` for ordinary asynchronous operations.

Better

C#

```
Func<Gift, Task> action =
    async gift =>
    {
        await SendReceiptAsync(
            gift);
    };
```

Then:

C#

```
await action(
    gift);
```

Async Delegate Flow

EXAMPLE / FLOW

```
Func<Gift, Task>
    ↓
Invoke callback
    ↓
Task returned
    ↓
await
    ↓
Completion / exception observed
```

Topic Boundary

We'll study:

EXAMPLE / FLOW

```
async

await

Task

Task<T>
```

deeply in Topic 27.

For now remember:

Asynchronous callback → usually `Func<..., Task>` rather than `Action<...>`

28. Delegate vs Interface

Suppose you need one behavior:

```
C#
Func<Gift, bool>
```

This can be simple and elegant.

But suppose the strategy requires:

```
EXAMPLE / FLOW
Validation

Calculation

Logging

Configuration

State

Multiple methods
```

then an interface may communicate the abstraction better.

Example:

```
C#
public interface IGiftEligibilityPolicy
{
    bool IsEligible(
        Gift gift);

    string GetReason(
        Gift gift);
}
```

Delegate Works Well When

The abstraction is fundamentally:

```
EXAMPLE / FLOW
One operation
```

Example:

```
C#
Func<Gift, bool>
```

Interface Works Better When

The collaborator has:

EXAMPLE / FLOW

```
Multiple related operations

State

Lifecycle

Configuration

A meaningful domain identity
```

29. Delegate vs Strategy Object

Delegate:

C#

```
Func<Gift, decimal>
```

works well for lightweight behavior selection.

Strategy interface:

C#

```
public interface IGiftFeeStrategy
{
    decimal Calculate(
        Gift gift);
}
```

may be better when:

EXAMPLE / FLOW

```
Different strategies have dependencies

Strategies have configuration

Behavior is complex

DI should resolve implementations
```

30. Delegate vs Local Function

Local function:

C#

```
decimal CalculateFee(
    Gift gift)
{
    return gift.TotalAmount
        *
        0.03m;
}
```

Lambda:

C#

```
Func<Gift, decimal> calculateFee = gift => gift.TotalAmount * 0.03m;
```

Use a local function when you primarily need:

EXAMPLE / FLOW

```
A named helper inside one method
```

Use a delegate/lambda when you need:

EXAMPLE / FLOW

```
A behavior value
to pass/store/return
```

31. Delegates Are Strongly Typed

Suppose:

C#

```
Action<Gift> callback;
```

This is valid:

C#

```
void Handle(
    Gift gift)
```

But not:

C#

```
void Handle(
    Campaign campaign)
```

unless variance rules establish compatibility.

The compiler checks delegate signature compatibility.

That is one of the core advantages over untyped function-pointer approaches.

32. Delegates Are Reference Types

Delegate instances are objects.

You can:

EXAMPLE / FLOW

```
Store them in variables

Pass them around

Compare them

Combine them

Return them

Place them in collections
```

They derive through the CLR delegate type system and ultimately inherit from `System.Delegate`.

33. Delegate Assignment


```C#
Action<Gift> first = LogGift;

Action<Gift> second = first;
```

Now both reference delegate objects representing compatible invocation behavior.

Combining delegates doesn't mutate the source delegates; it produces another delegate representing the combined invocation list.

34. Method Parameters as Behavior Injection

Consider:

```C#
public static void ProcessGifts(
    IEnumerable<Gift> gifts,
    Predicate<Gift> shouldProcess,
    Action<Gift> process)
{
    foreach (
        Gift gift
        in gifts)
    {
        if (
            shouldProcess(
                gift))
        {
            process(
                gift);
        }
    }
}
```

Call:

```C#
ProcessGifts(
    gifts,
    gift =>
        gift.Status
        == GiftStatus.Pending, gift => Console.WriteLine($"Processing {gift.GiftId}"));
```

This separates:

EXAMPLE / FLOW

```
Iteration algorithm
```

from:

EXAMPLE / FLOW

```
Selection behavior
```

and:

EXAMPLE / FLOW

```
Processing behavior
```

Fundraising Flow

EXAMPLE / FLOW

```
Gift collection
    ↓
Predicate<Gift>
    ↓
Should Gift be processed?
    ↓
Action<Gift>
    ↓
Perform operation
```

35. Functional Composition

Functions can be combined conceptually.

C#

```
Func<Gift, decimal> amountSelector = gift => gift.TotalAmount;

Func<decimal, decimal> feeCalculator = amount => amount * 0.03m;
```

Composition:

C#

```
Func<Gift, decimal> giftFee = gift => feeCalculator(amountSelector(gift));
```

Flow:

EXAMPLE / FLOW

```
Gift
    ↓
Amount selector
    ↓
decimal
    ↓
Fee calculator
    ↓
decimal fee
```

This style appears frequently with LINQ and functional APIs.

36. Delegate Factories

You can create specialized behavior dynamically.

C#

```
public static Predicate<Gift>
    CreateMinimumGiftPredicate(
        decimal minimum)
{
    return gift =>
        gift.TotalAmount >= minimum;
}
```

Usage:

```C#
Predicate<Gift> majorGift = CreateMinimumGiftPredicate(10_000m);
```

Now:

```C#
majorGift(
    gift);
```

applies the configured threshold.

37. Avoid Hidden Mutable Capture

Potentially confusing:

```C#
decimal threshold = 100m;

Predicate<Gift> filter = gift => gift.TotalAmount >= threshold;

// Later:
threshold = 10_000m;
```

Now behavior changed even though:

EXAMPLE / FLOW

```
filter
```

wasn't reassigned.

Sometimes that's intentional.

Sometimes it's a subtle bug.

Prefer immutable capture where possible.

Better Explicit Capture

```C#
decimal threshold = request.MinimumAmount;

decimal capturedThreshold = threshold;

Predicate<Gift> filter = gift => gift.TotalAmount >= capturedThreshold;
```

Or pass the value explicitly through your design rather than capturing mutable outer state.

38. Capturing `this`

Inside an instance method:

```C#
public Predicate<Gift>
    CreateFilter()
{
    return gift =>
        gift.TenantId
        == TenantId;
}
```

The lambda references:

```
C#
TenantId
```

from the current object.

Therefore it captures:

```
EXAMPLE / FLOW
this
```

indirectly.

That can keep the containing object alive as long as the delegate remains reachable.

Captured outer variables remain alive while required by reachable delegates.

Lifetime Concern

```
EXAMPLE / FLOW
Long-lived delegate
    ↓
captures service/object
    ↓
service/object remains reachable
    ↓
larger object graph may remain alive
```

This matters with:

```
EXAMPLE / FLOW
Static delegate storage

Events

Caches

Long-lived subscriptions
```

and connects strongly to Topic 23.

39. Delegate Performance

A direct method call is generally simpler than invoking through a delegate.

Delegates add:

```
EXAMPLE / FLOW
Indirection

Delegate object/state

Potential closure allocations

Potential target-object lifetime
```

Usually this overhead is tiny compared with:

EXAMPLE / FLOW

```
Database calls

HTTP calls

File I/O
```

so do not avoid delegates in normal application code.

Hot-Path Considerations

In highly optimized code, consider:

EXAMPLE / FLOW

```
Avoid unnecessary delegate creation

Avoid accidental captures

Reuse callbacks where appropriate

Use static lambdas for noncapturing behavior

Profile before optimizing
```

40. Security Considerations

Delegates represent **executable behavior**.

Do not treat external client input as though it can safely supply arbitrary executable delegates.

Clients should supply:

EXAMPLE / FLOW

```
Data

Identifiers

Configuration choices
```

and server code decides which trusted behavior to invoke.

Bad Concept

EXAMPLE / FLOW

```
HTTP client sends:
"Execute this delegate"
```

That isn't a normal safe API design.

Better

Client sends:

C#

```
{
  "receiptType": "Email"
}
```

Server maps:

EXAMPLE / FLOW

```
Email
    →
trusted EmailReceiptHandler
```

Sensitive Capture

Avoid long-lived callbacks capturing unnecessary sensitive values:

C#

```
string token =...;

Action callback = () => UseToken(token);
```

The captured token remains reachable as long as the closure remains reachable.

Keep sensitive data lifetimes narrow.

Callback Exceptions

If a callback is supplied by another internal component:

C#

```
process(
    callback);
```

decide explicitly whether:

EXAMPLE / FLOW

```
Callback exception should propagate

Callback should be isolated

Callback should be logged

Remaining callbacks should continue
```

Do not suppress exceptions blindly.

Common Mistake 1 — Calling Instead of Passing Method

Wrong:

C#

```
Action<Gift> callback = LogGift(gift);
```

Correct:

C#

```
Action<Gift> callback = LogGift;
```

Common Mistake 2 — Wrong Action/Func Choice

Wrong:

```C#
Action<Gift> calculate = gift => gift.TotalAmount;
```

`Action` returns:

```EXAMPLE / FLOW
void
```

Correct:

```C#
Func<Gift, decimal> calculate = gift => gift.TotalAmount;
```

Common Mistake 3 — Misreading Func

```C#
Func<Gift, Campaign, bool>
```

does **not** mean three inputs.

It means:

```EXAMPLE / FLOW
Input 1
    →
Gift

Input 2
    →
Campaign

Return
    →
bool
```

The final type parameter is the result.

Common Mistake 4 — Predicate vs Action

A predicate must answer:

```EXAMPLE / FLOW
true / false
```

This is wrong conceptually:

```C#
Predicate<Gift> logger =
    gift =>
    {
        Console.WriteLine(
            gift.GiftId);

        return true;
    };
```

It technically returns bool, but using a predicate purely for side effects is misleading.

Use:


```C#
Action<Gift>
```

for side-effect-only behavior.

Common Mistake 5 — Giant Lambda

Bad:

```C#
Func<Gift, bool> filter =
    gift =>
    {
        // 100 lines
    };
```

Extract meaningful behavior into a named method or collaborator.

Common Mistake 6 — Mutable Closure Surprise

```C#
int threshold = 10;

Func<int, bool> filter = value => value > threshold;

threshold = 100;
```

The delegate now compares against:

EXAMPLE / FLOW

```
100
```

not necessarily the value that existed when the delegate was created.

Common Mistake 7 — Loop Capture

Be careful when callbacks capture one variable that changes across iterations.

Create a per-iteration copy when you need a stable captured value.

Common Mistake 8 — Assuming Multicast Is Parallel

```C#
callbacks(
    gift);
```

does not automatically execute handlers concurrently.

Invocation is synchronous and ordered.

Common Mistake 9 — Ignoring Multicast Return Rules

If a multicast delegate returns a value, ordinary invocation gives you the final invoked method's return value.

Earlier results aren't automatically collected.

Common Mistake 10 — Assuming All Handlers Run After Exception

If one invocation-list method throws an uncaught exception:

EXAMPLE / FLOW

```
Later handlers do not run
```

during normal multicast invocation.

Common Mistake 11 — async Lambda Assigned to Action

C#

```
Action callback =
    async () =>
    {
        await DoWorkAsync();
    };
```

creates `async void`-style behavior.

For ordinary async callbacks use:

C#

```
Func<Task>
```

instead.

Common Mistake 12 — Assuming Lambda Is Always Delegate

A lambda can represent:

EXAMPLE / FLOW

```
Delegate
```

or:

EXAMPLE / FLOW

```
Expression tree
```

depending on target type.

This distinction becomes critical with LINQ providers.

Common Mistake 13 — Unnecessary Capture of this

C#

```
callbacks.Add(
    gift =>
        ProcessGift(
            gift));
```

may capture the current object when `ProcessGift` is an instance member.

If callback lives much longer than expected, the containing object may remain alive.

Common Mistake 14 — Using Delegate Where Interface Is Clearer

If behavior needs:

```
EXAMPLE / FLOW
Multiple methods

State

DI dependencies

Configuration
```

a delegate may become less readable than an interface.

Common Mistake 15 — Using Custom Delegates Everywhere

Don't create:

```
C#
GiftAction

GiftAction2

GiftFunc

GiftPredicate
```

when standard:

```
C#
Action<T>

Func<T,TResult>

Predicate<T>
```

already communicate the API clearly.

Best Practices

Use delegates when behavior itself must be:

```
EXAMPLE / FLOW
Passed

Stored

Returned

Selected dynamically
```

Use:

```
C#
Action
```

when there is no return value.

Use:

```C#
Func
```

when behavior returns a result.

Use:

```C#
Predicate<T>
```

when the semantic operation is a true/false test and the consuming API uses that delegate.

Remember:

EXAMPLE / FLOW

```
Last Func type
    →
return type
```

Prefer lambdas for small local behavior.

Prefer named methods for complex or reusable behavior.

Use descriptive parameter names:

```C#
gift =>
```

rather than meaningless names such as:

```C#
x =>
```

when domain clarity matters.

Avoid unnecessarily large lambdas.

Understand closure behavior before capturing mutable variables.

Use:

```C#
static
```

lambdas when you intentionally want to prevent capture.

Avoid long-lived delegates capturing large object graphs or sensitive values.

Use multicast delegates mainly for notification/callback scenarios where ordered synchronous invocation makes sense.

Remember that one failing multicast handler stops subsequent normal invocation.

Prefer:

```C#
Func<Task>
```

or:

```C#
Func<T, Task>
```

for asynchronous callbacks.

Avoid ordinary `async void` callbacks except appropriate event-handler scenarios.

Use custom delegate types when semantic naming improves a public API.

Use interfaces instead of delegates when the behavior represents a richer collaborator with multiple operations or state.

Do not confuse delegates with expression trees.

Do not let generic callback abstractions hide important security, tenant, or domain rules.

Practical Exercise 1 — Custom Delegate

Create:

```C#
public delegate decimal
    DiscountCalculator(
        decimal amount);
```

Then:

```C#
static decimal ApplyDiscount(
    decimal amount)
{
    return amount * 0.90m;
}
```

Assign:

```C#
DiscountCalculator calculator = ApplyDiscount;
```

Invoke:

```C#
decimal finalAmount = calculator(100m);
```

Result:

EXAMPLE / FLOW

```
90
```

Practical Exercise 2 — Action

```C#
Action<Gift> print = gift => Console.WriteLine(gift.GiftId);
```

Use:

```C#
print(
    gift);
```

Practical Exercise 3 — Func

```C#
Func<Gift, decimal> selectAmount = gift => gift.TotalAmount;
```

Use:

```C#
decimal amount = selectAmount(gift);
```

Practical Exercise 4 — Predicate

```C#
Predicate<Gift> completed = gift => gift.Status == GiftStatus.Completed;
```

Practical Exercise 5 — Method Argument

```C#
static void Execute(
    Gift gift,
    Action<Gift> action)
{
    action(
        gift);
}
```

Call:

```C#
Execute(
    gift,
    current =>
        Console.WriteLine(
            current.GiftId));
```

Practical Exercise 6 — Returning a Delegate

```C#
static Predicate<Gift>
    MinimumAmount(
        decimal minimum)
{
    return gift =>
        gift.TotalAmount >= minimum;
}
```

Usage:

```C#
Predicate<Gift> majorGift = MinimumAmount(5000m);
```

Practical Exercise 7 — Multicast Delegate

```C#
Action<Gift> handlers = LogGift;

handlers += SendReceipt;

handlers += PublishAudit;
```

Invocation:

```C#
handlers(
    gift);
```

Expected order:

EXAMPLE / FLOW

```
LogGift

SendReceipt

PublishAudit
```

Practical Exercise 8 — Remove Handler

```C#
handlers -= SendReceipt;
```

Now invocation runs:

EXAMPLE / FLOW

```
LogGift

PublishAudit
```

Practical Exercise 9 — Closure

```C#
decimal threshold = 1000m;

Predicate<Gift> filter = gift => gift.TotalAmount >= threshold;
```

Question:

What happens if:

```C#
threshold = 5000m;
```

before invocation?

Answer:

The lambda observes the captured variable's new value.

Practical Exercise 10 — Static Lambda

```C#
Func<Gift, decimal> selector = static gift => gift.TotalAmount;
```

This guarantees the lambda doesn't capture local variables or instance state.

Practical Exercise 11 — Async Callback

Prefer:

```C#
Func<Gift, Task> callback =
    async gift =>
    {
        await SendReceiptAsync(
            gift);
    };
```

Call:

```C#
await callback(
    gift);
```

Practical Exercise 12 — Collection Callback

```C#
gifts.RemoveAll(
    gift =>
        gift.Status
        == GiftStatus.Cancelled);
```

Here the lambda becomes a:

```C#
Predicate<Gift>
```

used by `List<Gift>`.

Exactly 25 Interview Questions

1. What is a delegate in C#?

A delegate is a type-safe object that represents one or more compatible methods.

Compatibility is based primarily on:

```EXAMPLE / FLOW
Parameter types

Return type
```

Delegates allow methods to be passed, stored, returned, and invoked later.

2. Why are delegates called type-safe function references?

Because the compiler verifies that the target method is compatible with the delegate's declared signature.

A delegate requiring:

```
C#
decimal Method(
    Gift gift)
```

cannot arbitrarily reference an unrelated method such as:

```
C#
void Method(
    Campaign campaign)
```

3. What is a method group?

Using a method name without invoking it:

```
C#
Action<Gift> action = ProcessGift;
```

gives the compiler a method group that can be converted to a compatible delegate.

Calling:

```
C#
ProcessGift(...)
```

would execute the method instead.

4. What is the difference between `Action` and `Func`?

`Action` represents behavior returning:

```
EXAMPLE / FLOW
void
```

while `Func` represents behavior returning a value.

For `Func`, the final generic argument is the return type.

5. What does `Func<Gift, decimal>` mean?

It represents a method that:

```
EXAMPLE / FLOW
Accepts
    →
Gift

Returns
    →
decimal
```

The final generic argument is always the result type.

6. What is Predicate<T>?

`Predicate<T>` represents:

EXAMPLE / FLOW

```
T → bool
```

and answers whether a value satisfies a condition.

For example:

C#

```
Predicate<Gift> isCompleted = gift => gift.Status == GiftStatus.Completed;
```

7. What is the difference between Predicate<T> and Func<T,bool>?

They have similar signatures but are different delegate types.

`Predicate<T>` is specifically named around testing a condition, while `Func<T,bool>` is the more general function form and is used extensively by LINQ.

8. What is a lambda expression?

A lambda is an anonymous function written with:

C#

```
=>
```

Example:

C#

```
gift =>
    gift.TotalAmount > 1000m
```

A lambda can convert to a compatible delegate or expression-tree type.

9. What is the difference between an expression lambda and statement lambda?

Expression:

C#

```
gift =>
    gift.TotalAmount
```

Statement:

C#

```
gift =>
{
    Console.WriteLine(
        gift.GiftId);

    return gift.TotalAmount;
}
```

The first consists of one expression; the second contains a block of statements.

10. What is a closure?

A closure occurs when a lambda or anonymous function uses variables from an enclosing scope.

Example:

```C#
decimal minimum = 1000m;

Predicate<Gift> filter = gift => gift.TotalAmount >= minimum;
```

The delegate retains access to the captured variable, which may extend that variable's lifetime.

11. Does a lambda capture the original value or the variable?

In ordinary captured-local scenarios, it captures the variable/state needed by the closure.

If that variable later changes, the lambda can observe the changed value.

12. What is a static lambda?

A lambda marked:

```C#
static
```

cannot capture local variables or instance state from the enclosing context.

Example:

```C#
Func<int, int> square = static value => value * value;
```

This can help prevent unintended closures.

13. What is a multicast delegate?

A multicast delegate has multiple methods in its invocation list.

Example:

```C#
action = First;

action += Second;
```

Calling `action()` invokes the methods synchronously in invocation-list order.

14. What happens if a multicast delegate has a return value?

Ordinary invocation returns the value from the last method invoked in the invocation list.

Earlier return values are not automatically collected.

15. What happens if one multicast delegate handler throws an exception?

If the exception isn't handled inside that handler, it propagates to the delegate caller and later handlers in the invocation list aren't invoked.

16. How do you add and remove delegate methods?

Use:

```
C#
+=
```

to add and:

```
C#
-=
```

to remove.

Example:

```
C#
handlers += LogGift;

handlers -= LogGift;
```

If duplicate entries exist, a removal removes one matching occurrence.

17. What is delegate covariance?

A compatible method may return a type more derived than the delegate requires.

For example, a delegate returning `Entity` may reference a compatible method returning `Gift` because `Gift` derives from `Entity`.

18. What is delegate contravariance?

A compatible method may accept a less-derived parameter type than the delegate requires.

A method capable of accepting every `Entity` can safely accept a `Gift`.

19. What do `in` and `out` mean for `Action` and `Func`?

`Action<T>` uses a contravariant:

```
EXAMPLE / FLOW
in T
```

because it consumes values.

`Func<T,TResult>` uses:

```
EXAMPLE / FLOW
in T
```

for its input and:

```
EXAMPLE / FLOW
out TResult
```

for its result.

20. What is the difference between a delegate and an event?

A delegate represents callable behavior directly.

An event builds on delegates but restricts outside code primarily to subscription and unsubscription, so outside consumers can't normally raise or replace the publisher's event.

This is covered deeply in Topic 23.

21. When should you create a custom delegate instead of using Action or Func?

DELEGATES / C#

Use a custom delegate when the semantic name matters or the signature doesn't fit the standard built-in delegate families conveniently.

For example:

```
C#
public delegate bool
    GiftEligibilityRule(
        Gift gift,
        Campaign campaign);
```

may communicate intent better than:

```
C#
Func<Gift, Campaign, bool>
```

in a public domain API.

22. What is the difference between a delegate and an expression tree?

A delegate represents executable behavior.

An:

```
C#
Expression<Func<T,bool>>
```

represents a data structure describing code that can be inspected or translated.

The same lambda syntax can produce either depending on its target type.

23. Why should ordinary asynchronous callbacks use Func<Task> instead of Action?

An async lambda assigned to a void-returning `Action` behaves as `async void`, so the caller can't await completion and normal task-based exception propagation is lost.

For ordinary asynchronous callbacks prefer:

```
C#
Func<Task>
```

or:

```
C#
Func<T, Task>
```

`async void` should primarily be reserved for event-handler scenarios.

24. When should you use a delegate instead of an interface?

A delegate is ideal when the abstraction is essentially:

EXAMPLE / FLOW

```
One operation
```

such as:

An interface is usually clearer when the collaborator contains multiple related operations, state, configuration, dependencies, or lifecycle concerns.

25. How are delegates used in real .NET applications?

They appear throughout:

EXAMPLE / FLOW

```
LINQ filters and projections

Collection callbacks

Sorting/comparison

Events

ASP.NET Core configuration callbacks

Async callbacks

Strategy behavior

Factories

Testing hooks
```

The key design principle is that delegates allow **behavior itself to become a strongly typed value** that can be passed around and invoked later.

Quick Revision Table

Concept	Meaning
Delegate	Type-safe method reference
Method group	Method name used without invocation
`Action`	No inputs, no return
`Action<T>`	Input, no return
`Func<TResult>`	No input, returns result
`Func<T,TResult>`	Input → result
`Predicate<T>`	`T → bool`
Lambda	Anonymous function using `=>`
Expression lambda	Lambda with one expression
Statement lambda	Lambda with statement block
Closure	Lambda captures outer state
Static lambda	Lambda that cannot capture surrounding local/instance state
Callback	Behavior passed for later invocation
Multicast delegate	Delegate with multiple methods

Concept	Meaning
Invocation list	Methods contained by multicast delegate
Covariance	More-derived return allowed
Contravariance	Less-derived input allowed
`Func<Task>`	Common async callback
Expression tree	Inspectable representation of code
Event	Encapsulated delegate-based notification mechanism

Complete Fundraising Example

Consider a gift-processing pipeline where we want configurable:

EXAMPLE / FLOW

```
Validation

Fee calculation

Completion callback

Async receipt delivery
```

Instead of hard-coding every behavior, we can use delegates where the behavior is truly a single operation.

Gift

C#

```
public sealed class Gift
{
    public Gift(
        long giftId,
        long tenantId,
        decimal totalAmount)
    {
        if (
            giftId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(giftId));
        }

        if (
            tenantId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }

        if (
            totalAmount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(totalAmount));
        }

        GiftId = giftId;
```

```
        TenantId = tenantId;

        TotalAmount = totalAmount;

        Status = GiftStatus.Pending;
    }

    public long GiftId
    {
        get;
    }

    public long TenantId
    {
        get;
    }

    public decimal TotalAmount
    {
        get;
    }

    public GiftStatus Status
    {
        get;
        private set;
    }

    public void Complete()
    {
        if (
            Status
            !=
            GiftStatus.Pending)
        {
            throw new InvalidOperationException(
                "Only pending gifts can be completed.");
        }

        Status = GiftStatus.Completed;
    }
}
```

Processor

C#

```
public sealed class GiftProcessor
{
    public async Task<decimal>
        ProcessAsync(
            Gift gift,
            Predicate<Gift> validator,
            Func<Gift, decimal>
                feeCalculator,
            Action<Gift> onCompleted,
            Func<Gift, Task>
                sendReceiptAsync)
    {
        ArgumentNullException.ThrowIfNull(
            gift);

        ArgumentNullException.ThrowIfNull(
```

```
            validator);

        ArgumentNullException.ThrowIfNull(
            feeCalculator);

        ArgumentNullException.ThrowIfNull(
            onCompleted);

        ArgumentNullException.ThrowIfNull(
            sendReceiptAsync);

        if (
            !validator(
                gift))
        {
            throw new InvalidOperationException(
                "Gift validation failed.");
        }

        decimal fee = feeCalculator(gift);

        gift.Complete();

        onCompleted(
            gift);

        await sendReceiptAsync(
            gift);

        return fee;
    }
}
```

Configure Behaviors

Validation:

```
C#
Predicate<Gift> validator = gift => gift.TotalAmount > 0m;
```

Fee calculation:

```
C#
Func<Gift, decimal> feeCalculator = gift => gift.TotalAmount * 0.025m;
```

Completion callbacks:

```
C#
Action<Gift> onCompleted = gift => Console.WriteLine($"Gift {gift.GiftId} completed.");

onCompleted += gift => Console.WriteLine($"Amount: {gift.TotalAmount:C}");
```

Async receipt:

```
C#
Func<Gift, Task> sendReceiptAsync =
    async gift =>
    {
        await Task.Delay(
            100);
```

```
        Console.WriteLine(
            $"Receipt sent for Gift {gift.GiftId}.");
    };
```


Execute

C#

```
Gift gift =
    new(
        giftId:
            1001,
        tenantId:
            1,
        totalAmount:
            1000m);

GiftProcessor processor = new();

decimal fee =
    await processor.ProcessAsync(
        gift,
        validator,
        feeCalculator,
        onCompleted,
        sendReceiptAsync);
```

Flow

EXAMPLE / FLOW

```
Gift
    ↓
Predicate<Gift>
    ↓
Valid?
    ├─ No → Exception
    └─ Yes
          ↓
Func<Gift, decimal>
          ↓
Calculate fee
          ↓
Gift.Complete()
          ↓
Action<Gift>
          ↓
Completion callbacks
          ↓
Func<Gift, Task>
          ↓
await Receipt
          ↓
Return fee
```

What Each Delegate Means

C#

```
Predicate<Gift>
```

means:

EXAMPLE / FLOW

"Should this Gift be accepted?"

C#

```
Func<Gift, decimal>
```

means:

EXAMPLE / FLOW

"Calculate something from this Gift."

C#

```
Action<Gift>
```

means:

EXAMPLE / FLOW

"Perform a synchronous side effect."

C#

```
Func<Gift, Task>
```

means:

EXAMPLE / FLOW

"Perform asynchronous work
and allow caller to await it."

When This Design Would Become Too Delegate-Heavy

Suppose processing later requires:

EXAMPLE / FLOW

```
Database access

Configuration

Complex validation rules

Multiple methods

External APIs

Caching

State

Metrics
```

Then instead of having a constructor like:

C#

```
GiftProcessor(
    Predicate<Gift>,
    Func<Gift, decimal>,
    Action<Gift>,
    Func<Gift, Task>,
```

```
    Func<Gift, Task<bool>>,
    ...)
```

DELEGATES / C#

you would probably move toward meaningful interfaces such as:

```
C#
IGiftValidator

IGiftFeeCalculator

IGiftReceiptSender
```

Delegates are excellent for small units of behavior.

Interfaces are usually better for richer collaborators.

Final Delegate Decision Flow

```
EXAMPLE / FLOW
Need to represent behavior
        ↓
Does behavior return a value?
```

No:

```
EXAMPLE / FLOW
Action
```

Yes:

```
EXAMPLE / FLOW
Func
```

If semantic meaning is:

```
EXAMPLE / FLOW
Test T and return bool
```

then:

```
EXAMPLE / FLOW
Predicate<T>
```

may fit.

Named Method or Lambda?

```
EXAMPLE / FLOW
Behavior small and local?
    ↓
Lambda
```

```
EXAMPLE / FLOW
Behavior complex/reused?
    ↓
Named method
```

Standard or Custom Delegate?

EXAMPLE / FLOW

```
Signature meaning obvious?
    ↓
Action / Func
```

EXAMPLE / FLOW

```
Domain semantics deserve name
or unusual signature required?
    ↓
Custom delegate
```

Sync or Async?

EXAMPLE / FLOW

```
Synchronous no result
    →
Action<T>
```

EXAMPLE / FLOW

```
Synchronous result
    →
Func<T,TResult>
```

EXAMPLE / FLOW

```
Asynchronous no result
    →
Func<T,Task>
```

EXAMPLE / FLOW

```
Asynchronous result
    →
Func<T,Task<TResult>>
```

Delegate or Interface?

EXAMPLE / FLOW

```
One lightweight operation
    →
Delegate
```

EXAMPLE / FLOW

```
Rich collaborator
with multiple behaviors/state/dependencies
    →
Interface
```

Closure Decision

EXAMPLE / FLOW

```
Lambda needs outer variable?
        ↓
```

```
Yes
 *  →
Capture occurs
        ↓
Ask:
Does this create hidden lifetime,
allocation, or mutation concerns?
```

If capture is unnecessary:

EXAMPLE / FLOW

```
Use static lambda
or pass value explicitly
```

Complete End-to-End Mental Model

EXAMPLE / FLOW

```
Method
    ↓
Method reference
    ↓
Delegate
    ↓
Store / pass / return
    ↓
Invoke later
```

Built-in forms:

EXAMPLE / FLOW

```
Action
    →
Do something
```

EXAMPLE / FLOW

```
Func
    →
Calculate something
```

EXAMPLE / FLOW

```
Predicate
    →
Ask true/false question
```

Lambda:

EXAMPLE / FLOW

```
Short anonymous implementation
of delegate behavior
```

Closure:

EXAMPLE / FLOW

```
Lambda remembers outer state
```

Multicast:

EXAMPLE / FLOW

```
One delegate
    ↓
Multiple ordered methods
```

Async:

EXAMPLE / FLOW

```
Func<T, Task>
    ↓
Awaitable callback
```

LINQ:

EXAMPLE / FLOW

```
Collection
    +
Func<T,...>
    ↓
Reusable algorithms
```